Jenkins Installation Report

By Cassidy Johnson

Summary:
In this document, I will be showing a step by step example of installing and configuring a Jenkins server in Amazon Web Services. Jenkins is an open source application for continuous integration. CI (closely related to CD, continuous development) is a fairly recent idea in software development that developers on teams shouldn't push their code to a repository without seeing how it integrates with other developers' features that may be concurrently developed.

Consider this scenario: Say it's a Friday afternoon and two developers (named A and B) are working together on a project with tens of other developers. Developer A has been working all week on a feature that they're really excited about and they just finished it. It's been reviewed by other developers, but developer B wasn't on the review team and doesn't know about the specifics of developer A's feature. So, developer A commits and pushes their new feature to the master branch of the Github repository that the whole team shares. Similarly, developer B has been working on another awesome feature and they also wrap up their work on it on Friday afternoon. Developer B will then notice that there are changes on the master branch that haven't been pulled into their feature branch yet (those changes are the changes made by developer A). B will pull the changes in and they may notice that those changes conflict with their new feature. Maybe it's a problem of resources, like both developers making something that is multi-threaded and in combination, their two features exceed the thread limit for the system. Or maybe they each changed the same section of the code but went in two different directions with it. Or maybe there is some other reason why developers A and B made features that simply don't mesh well. This means that developer B's time was wasted in making that feature since they can't push it without breaking the source tree. Bummer.

However, with CI, developer A could have been steadily integrating their code into the repo in real time and developer B could see those changes as they're happening. Now this isn't really as continuous as the name suggests since most configurations have developers' code integrated a few times a day, not actually continuously. But it does enable teams to find problems before they grow out of control. It saves people time and improves communication within a team. According to jenkins.io, "Jenkins can be used as a simple CI server or turned into the continuous delivery hub for any project." A CD (continuous delivery) server is a similar concept, except instead of pushing code to a server that your team of developers see, you push features to a server that your users/clients will see so that they get features and bug fixes shipped to them frequently. In this document, we'll learn how to install and configure Jenkins for CI.

Installation:
We will be installing the open source Jenkins software in a 64 bit Ubuntu environment. There are three main installation steps, outlined as follows.

Overview
Step 1: Get an Ubuntu server up and running in AWS
Step 2: Grab some dependencies (Java)
Step 3: Install Jenkins

Step 1
We will be using an Ubuntu server with a desktop as our Jenkins server in AWS. In the AWS EC2 portal, Launch an Instance by searching for and selecting "Ubuntu Server 20.04 LTS (HVM), SSD Volume Type". Use the following details to configure the server:

- t2.micro with 1vGB
- put this instance in the **public subnet** of Tiger-VPN
- add 10 GB of storage
- create a Name tag and call it "Jenkins Ubuntu Server"
- make a new security group called "Jenkins Security Group" and allow all TCP traffic **from any address** to connect to port 8080 (this is the port that the Jenkins Web app is exposed on) and SSH (port 22) **from any address**. Make no outbound rules. This means that anyone can try to SSH into the server if they have the correct key, and anyone will be able to try to connect to the Jenkins web app. We will configure some auth requirements later to protect the information on our server.
- Use the pre-existing .pem file we've been using for every instance in this class

If you want to get extra fancy, you can look into putting the Jenkins server in the *private* subnet of the Tiger-VPC and set up a load balancer for it or try to VPN into it. However, I think this is unnecessary. As long as we guard our SSH key and follow the Jenkins security protocols detailed later, our server will be safe.

Now that we have a basic Ubuntu server up, we need to grab some dependencies using the package manager, apt. SSH into the Linux server using the private key .pem file so we can begin downloading dependencies.

```
$ ssh -i [NAME OF YOUR PEM FILE] ubuntu@[PUBLIC IP OF JENKINS SERVER]
```

Step 2
Java is a necessary dependency for Jenkins to run properly. If you're curious about a bit of history behind Jenkins, you may find it interesting that the creator of Jenkins worked for Sun Microsystems and developed the tool to aid in his workflow at Sun. It got so popular that there was a custody battle over it that ultimately resulted in a name change. As its roots are in Sun Microsystems, it needs Java to run. As a rule of thumb, we shouldn't start installing things with apt until we update it. Proceed with the command below to download a JDK setup:

```
$ sudo apt update
$ sudo apt-get install default-jdk
```

NOTE: if you have a preferred jdk version, feel free to use that one! However, Jenkins is picky about what it supports, so steer clear of Java versions 9, 10, and 12. For me, the default was 11. See the java requirements on the Jenkins docs for more information.

Ensure java was properly installed.

```
$ java --version
```

This command should spit out some information about your java version, which for me was 11. If it returns any error code, java didn't properly download. Check this documentation for more specific instructions to follow if you weren't able to successfully install java or if you want to peruse the different versions you can choose from.

Now, we have a java developer kit at the ready. Let's start downloading and configuring Jenkins!

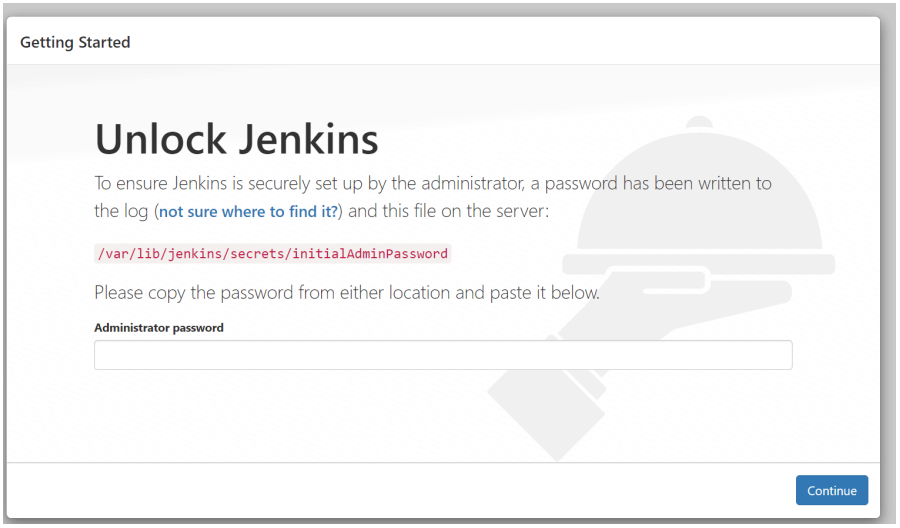
Step 3

Thanks to our handy dandy package manager "apt", installing Jenkins is quite simple. Please do the following in your SSH session:

```
$ wget -q -O - https://pkg.jenkins.io/debian-stable/jenkins.io.key |
sudo apt-key add -
$ sudo sh -c 'echo deb https://pkg.jenkins.io/debian binary/ > \
    /etc/apt/sources.list.d/jenkins.list'
$ sudo apt-get update # DON'T SKIP THIS
$ sudo apt-get install jenkins
```

Those first few commands may be unfamiliar, so let's go through them one by one. The first line that says "wget" is giving an HTTP request to the url and then adding the key it got from that request to our package manager. Without this step, we won't be able to get Jenkins from the package manager, apt. The next line, which starts with "sudo sh -c" is a shell command that grabs the debian binary for Jenkins and puts it in the list of sources for apt. This is also necessary for apt to be able to manage Jenkins in the future. This is easier for us in the long run than manually downloading Jenkins because apt will be able to do updates for us as they're released. Then, we update apt since we've just made changes to its source list and finally install Jenkins.

Now, every piece of software that we need to have a Jenkins server is officially installed. In order to test it out and see our Jenkins homepage, use the web browser of your choice in your local machine and navigate to "http://[PUBLIC-IP-OF-JENKINS-SERVER]:8080". You will be greeted with the "unlock Jenkins" page. Since it's not configured and this is a brand new installation, you will need to prove to Jenkins that you are truly an admin on the machine that installed Jenkins.

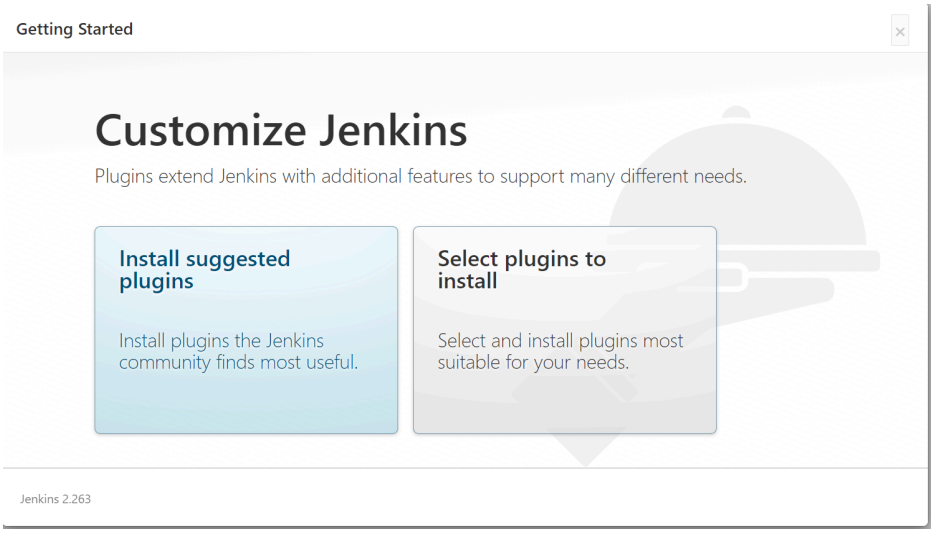


Copy the shared secret from the Jenkins log file as shown and paste it into the Administrator Password prompt:

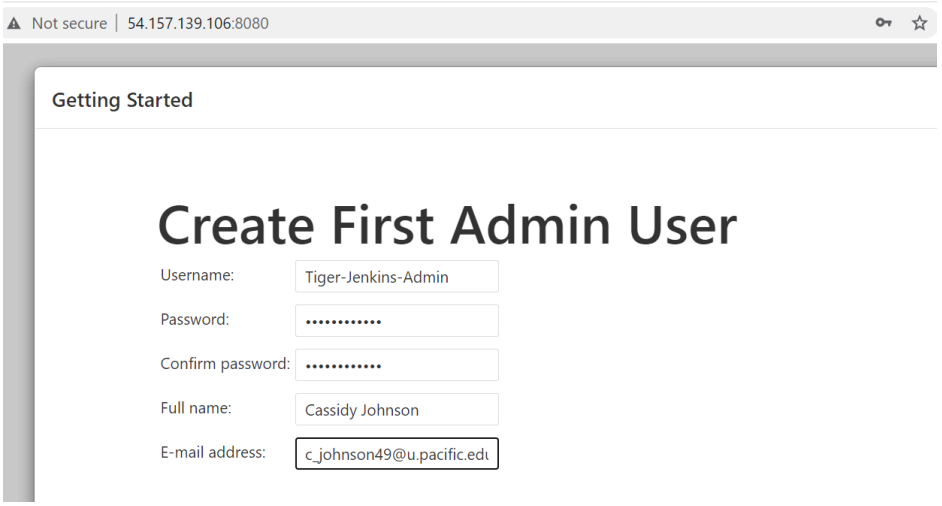
```
$ sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

That's it for the installation, on to the configuration!

Configuration:
We will go through the initial config steps in the web app with minimal changes to the defaults. First, install the suggested plugins. We may go back later and add our own, depending on the needs of Tiger Enterprises, but we can simply use what is given to us for now.



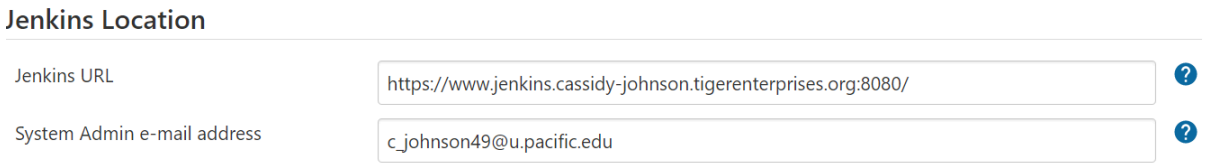
Next, we will need to make an account for the Jenkins administrator (that's you!) so choose a strong password.



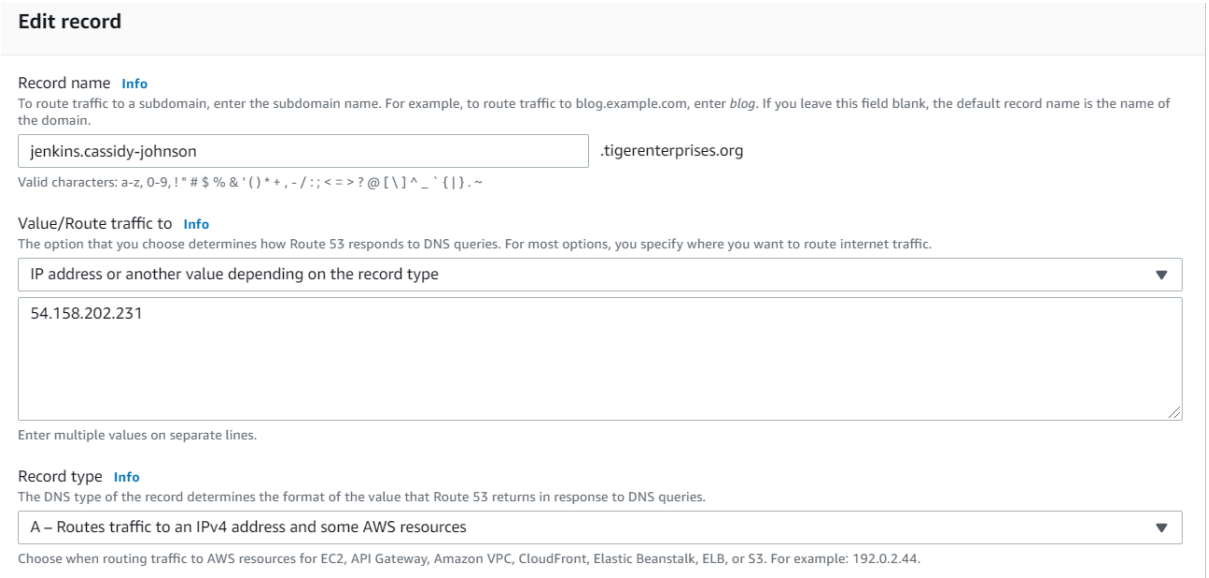
It is important here to use a secure password. If someone guesses your password, they can change all the Jenkins configuration for your entire team. After you've made your admin account, finish up the configuration without changing any defaults. Now, you should be able to access the Jenkins dashboard with the big "**Welcome to Jenkins**" header! One more quick setting before we start the security process. Click on the Manage Jenkins option and go to Configure System. Scroll to an option that says "system admin email address". Type your email in there, so that if you ever set up system monitoring of your Jenkins server in the future, you will be emailed when anomalies pop up. That's a bit out of the scope of this document, but the system should still list your email as that of the administrator for the purposes of this setup.

Right above the email address box, it says "Jenkins URL". Let's put Jenkins inside the Tiger Enterprises domain. Remember your domain name from previous labs? Enter "jenkins."

followed by that domain name here and remember to include the port number as well.



In the AWS Console, you will need to add this record to your hosted zone so that it is configured with the same DNS settings as the vpn and the main website for Tiger Enterprises. Find the Route 53 page in the AWS console and click on your hosted zone (there should be only 1). In this hosted zone, you'll see a list of a few records, including www.student-name.tigerenterprises.org and .vpn.student-name.tigerenterprises.org. Add a new record for jenkins.student-name.tigerenterprises.org. Unfortunately, we can't include the port number in the IP address that the domain name should map to, so we'll have to type in the port number when we connect to the web page, just like you typed into the Jenkins URL in the screenshot above.



Use the public IP of the Jenkins server for the IP address and make it a type A record. Now, reload your jenkins page with the new url. For now, don't type in "https", even though that's what we told Jenkins its URL would be in the configuration page. We haven't yet set up HTTPS for Jenkins, but that's coming soon! At least now there's no more having to find the public IP address of your Jenkins server. If you repeat the initial steps of lab 8, you can automate it so that the record keeps up with the changing IP address anytime you log out of your Jenkins server.

Security:
Next, we'll make some configuration changes in the Jenkins' config file for added security, which is found at /etc/default/jenkins. At the time of installation, Jenkins will have a simple and insecure configuration. We're going to change that.

Overview
Step 1: Logging and HTTPS
Step 2: Authentication
Step 3: Authorization

Logging and HTTPS
Return to your SSH session from earlier and open /etc/default/jenkins in your favorite text editor. You'll need to use "sudo" to accomplish this. Below is an abbreviated version of the config file that highlights the parts we are interested in changing. Specifically, we care about the logs and the http/https arguments that get passed to Jenkins on startup.

```
# defaults for Jenkins automation server

...

# log location.  this may be a syslog facility.priority
JENKINS_LOG=/var/log/$NAME/$NAME.log
#JENKINS_LOG=daemon.info

# Whether to enable web access logging or not.
# Set to "yes" to enable logging to /var/log/$NAME/access_log
JENKINS_ENABLE_ACCESS_LOG="no"

...

# Port for HTTP connector (default 8080; disable with -1)
HTTP_PORT=8080


# servlet context, important if you want to use apache proxying
PREFIX=/$NAME

# arguments to pass to jenkins.
# --javahome=$JAVA_HOME
# --httpListenAddress=$HTTP_HOST (default 0.0.0.0)
# --httpPort=$HTTP_PORT (default 8080; disable with -1)
# --httpsPort=$HTTP_PORT
# --argumentsRealm.passwd.$ADMIN_USER=[password]
# --argumentsRealm.roles.$ADMIN_USER=admin
# --webroot=~/.jenkins/war
# --prefix=$PREFIX
```

```
JENKINS_ARGS="--webroot=/var/cache/$NAME/war --httpPort=$HTTP_PORT"
```

Edit the line that says “JENKINS_ENABLE_ACCESS_LOG” to say “yes” instead of “no”. Change the HTTP port to be -1, which tells Jenkins you don’t want to use HTTP at all and add a line that defines the HTTPS_PORT to be 8080. Your config file will now look something like this:

```
# defaults for Jenkins automation server

...

# log location.  this may be a syslog facility.priority
JENKINS_LOG=/var/log/$NAME/$NAME.log
#JENKINS_LOG=daemon.info

# Whether to enable web access logging or not.
# Set to "yes" to enable logging to /var/log/$NAME/access_log
JENKINS_ENABLE_ACCESS_LOG="yes"

...

# Port for HTTPS connector
HTTP_PORT=-1
HTTPS_PORT=8080

# servlet context, important if you want to use apache proxying
PREFIX=/$NAME

# arguments to pass to jenkins.
# --javahome=$JAVA_HOME
# --httpListenAddress=$HTTP_HOST (default 0.0.0.0)
# --httpPort=$HTTP_PORT (default 8080; disable with -1)
# --httpsPort=$HTTP_PORT
# --argumentsRealm.passwd.$ADMIN_USER=[password]
# --argumentsRealm.roles.$ADMIN_USER=admin
# --webroot=~/.jenkins/war
# --prefix=$PREFIX

JENKINS_ARGS="--webroot=/var/cache/$NAME/war --httpPort=${HTTP_PORT}
--httpsPort=${HTTPS_PORT}"
```

Take careful note of the last line of the file to make sure your JENKINS_ARGS are all correct and match my example. These arguments are used to start Jenkins and any mistake in the arguments will result in your Jenkins web app being inaccessible. As of now, if we restart Jenkins, it won’t

work because we told it to use HTTPS but we haven't provided any certificates to make that happen. In order to finish the HTTPS settings, we need to download a certificate from Let's Encrypt, similar to what we did in lab 6. The lab 6 documentation has plenty of detail, but I'll summarize below as it is relevant to our Jenkins security configuration. First, we'll want to make sure that snap is updated. Snap is another package manager, like apt, and it will be our package manager of choice to get access to a certificate creating service.

```
$ sudo snap refresh core
$ sudo snap install --classic certbot
```

Once certbot is downloaded, make a symlink so that it can run as an application managed by snap.

```
$ sudo ln -s /snap/bin/certbot /usr/bin/certbot
```

Then, ask certbot to make a certificate.

```
$ sudo certbot certonly --standalone
```

You will be prompted with a few questions, most of which are up to your preferences. When asked, give the full domain name of your Jenkins server as you entered it into your Route 53 record, "jenkins.student-name.tigerenterprises.org". Don't include "https://www." or the port number.

Now, you should have multiple .pem files in the */etc/letsencrypt/live/jenkins.student-name.tigerenterprises.org/* directory. These are certificate files that we need to convert into the .jks format so that Jenkins will recognize them. Run the following commands and when prompted, input a password that you will remember and nobody else will know.

```
$ sudo su
# openssl pkcs12 -inkey privkey.pem -in cert.pem -export -out
keys.pkcs12
# keytool -importkeystore -srckeystore keys.pkcs12 -srcstoretype pkcs12
-destkeystore /var/lib/jenkins/jenkins.jks
```

There are two different passwords that you'll be asked for and you should make sure they are the same password. Whatever that password is, we are going to put it into the Jenkins config file. Remember the config file at /etc/default/jenkins? Open that back up again and add two additional options in your Jenkins arg on the last line of the file:

```
...
```

```
# Port for HTTPS connector
HTTP_PORT=-1
HTTPS_PORT=8080
SSL=/var/lib/jenkins/jenkins.jks  # add this line
password=your-password  # place the password you chose here

JENKINS_ARGS="--webroot=/var/cache/$NAME/war --httpPort=-1
--httpsPort=8080 --httpsKeyStore=${SSL}
--httpsKeyStorePassword=${password}"
```

All other lines of the file should stay the same. You only need to edit the last line and add the SSL and password variables.

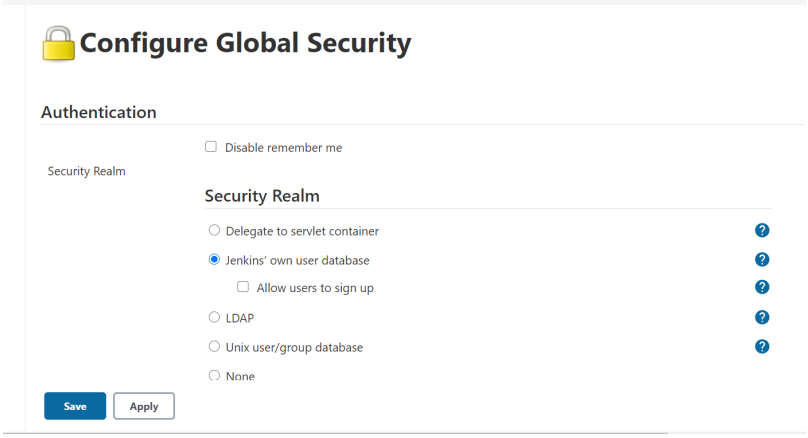
```
# jenkins service restart
```

Make sure to restart the Jenkins service and reload the webpage anytime you make configuration changes to ensure it went through okay. Only this time, when we reload the page, we should use the DNS name instead of the ip address and check that the lock pops up next to the URL, showing us that our page is encrypted and using the HTTPS protocol.

If you'd like to check that the log configurations we did earlier worked, go to the Jenkins dashboard. By checking out the System Log in the Manage Jenkins page, you can see all the logs we enabled. As a note, I'll mention that the logs have different statuses. A status less serious than "Info" isn't logged, but "Error" and "Info" statuses are. You can get really fancy with the log recorders, but we'll leave the logs as a basic tool to search through if something goes wrong. You can also find them at */var/lib/jenkins/log* if you prefer to SSH into the Jenkins server to view the logs instead of viewing them in the Web App.

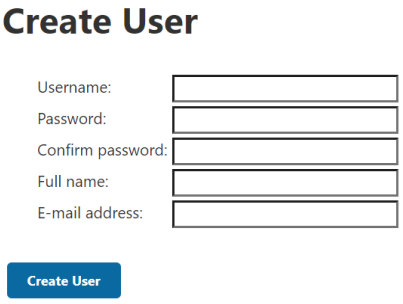
Authentication

The next step is to set up authentication for other users. After all, we did allow anyone to connect to port 8080 in our Jenkins Security Group, but it's likely we don't want just anyone to be able to change or even see our builds. Return to the Jenkins dashboard and click "Manage Jenkins" and scroll down to "Configure Global Security". This is the page we use to define who can access Jenkins (authentication requirements) and what those users can do (authorization settings).



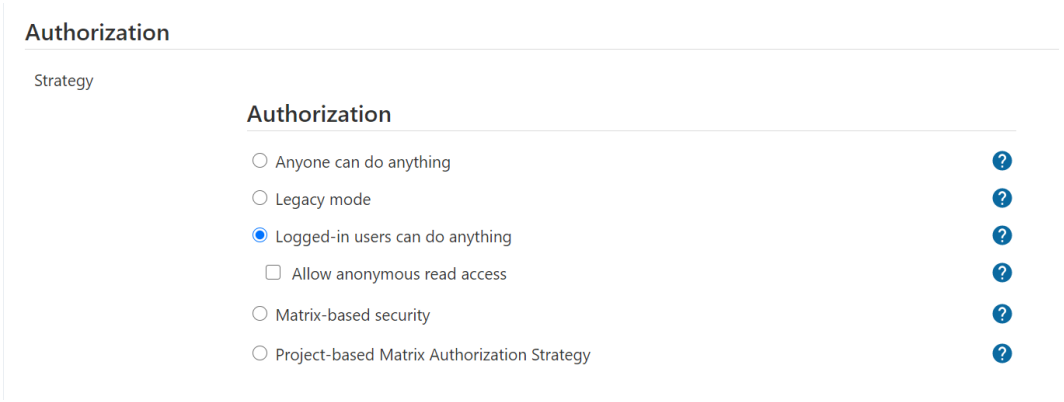
You'll notice that under the Security Realm options, the automatic settings are that Jenkins has its own user database for authentication. With this setting, the administrator (you) will have to add accounts one by one. The benefit of this is that you can choose who gets access and the default is that a user does not have access until you manually change that. The downside is the amount of effort it would require to add each user one by one. In large organizations, that becomes quite unwieldy.

Let's keep the "Jenkins' own user database" option and try it for ourselves by adding a user manually since we don't have hundreds of people to add. In order to add a user, return to the Manage Jenkins page and choose Create User. You will have to manually enter their name, email address, and a password. I recommend using a personal email account to add yourself as a user, but a less privileged user than your administrative account. That way, you can log out of your admin account and try logging in with your personal email address to ensure it works. Note that our next step, which is authorization, is very important. Imagine if anyone could create a Jenkins account and then add users to it. This is a setting that we want to make sure only Jenkins administrators have.



<u>Authorization</u>

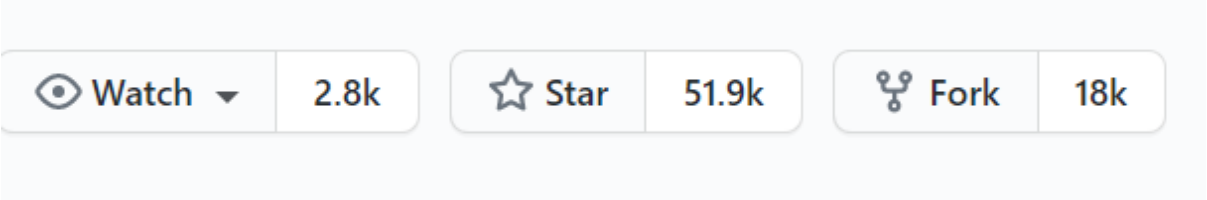
Make sure you're signed into Jenkins as the administrative account again. Next, we must handle the authorized actions for different user groups.



For this, we want to utilize Matrix Based security. Users who are logged in should be able to read everything, create configure, and build jobs. I've chosen not to allow every authenticated user to delete or move builds, but that really depends on the organization. Some teams may be so small and tight knit that everyone is trusted to manipulate builds as necessary. Some teams are large and there should be only a few selected users who can delete the builds to avoid losing build history. Below is a basic matrix based security setup for Jenkins:

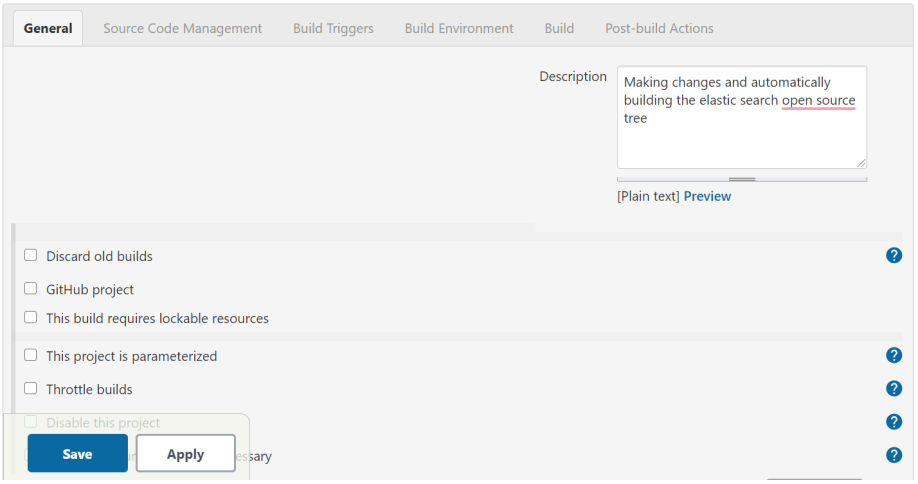
Make sure to **give yourself all permissions** since you're the administrator.

Using Jenkins:
Jenkins is now completely set up and secured! Now let's play around with it. To show a real world example, let's fork the ElasticSearch open source repo on Github. If you're interested, ElasticSearch is a tool in a larger stack of monitoring and data analysis tools. Once data has been gathered, ElasticSearch helps you parse through it all and find patterns. If you have a Github account, press the "Fork" button in the top right corner.

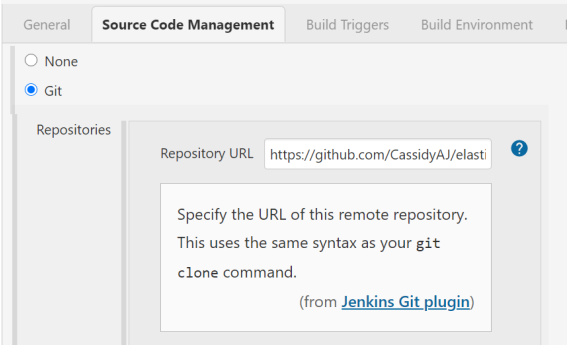


This will essentially make a local copy of the repo that you can change without affecting the source tree that everyone else is using. This will be our source for the Jenkins builds.

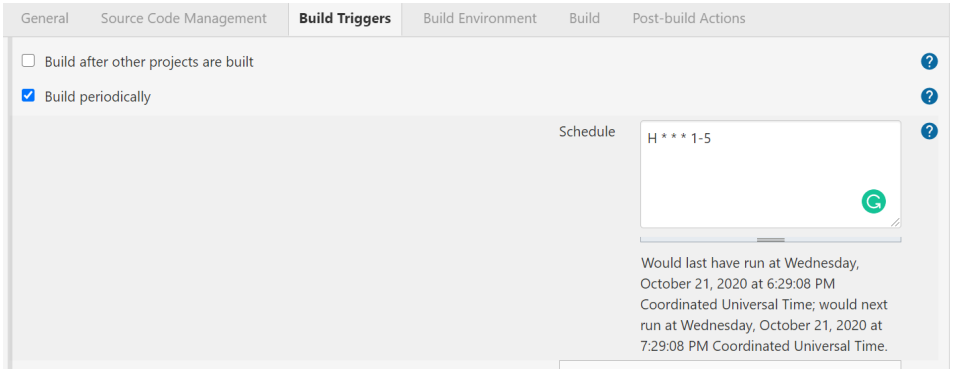
In the Jenkins dashboard, click New Item, then FreeStyle to make a new project. After you name the project and select "Git" as the source, you then have tons of options for configure the project, as shown here:



Add a description and leave all the boxes unchecked for now. We may come back later and check the "Discard old builds" option. I don't want to do that for now because old builds act as a good history of what went wrong when we run into broken builds. Continue to scroll through the options and pick what is right for your project. In my case, I chose to add a Git repository (the one I have in my profile now that I forked the ElasticSearch repo). If you're unsure of the url to include, it's the same you would use to Clone a repo. It should end in ".git".



Once the git repo is added, we have one more important step: telling Jenkins how often to build the repo. I chose hourly every day from Monday to Friday. This will obviously stop if the server is down, so it won't really happen that frequently since I won't be keeping my AWS console open with the Jenkins server running all day. The format is mostly the same as making a Cron job. In fact, you can choose the option to have a script trigger builds if you want to make your own Cron job for Jenkins. I found using the Web app easier.



Press "Apply" to apply your changes. That's it! You've made your first build in Jenkins! The ElasticSearch source tree will build every hour on weekdays and you can sign into the dashboard to view if the build succeeded or failed. As this document is truly about installing and configuring Jenkins, I've chosen to keep the example short. A longer explanation of using Jenkins can be found here.

Further References:

1. Jenkins Installation
2. Jenkins Installation on AWS
3. Jenkins Security Configuration
4. Jenkins Official Docs
5. Jenkins HTTPS
6. Let's Encrypt Docs for Ubuntu 20.04